Easy Guide to Writing IAM Policies in AWS

This guide explains the important parts of IAM policies in simple words.Writing IAM policies can seem confusing, but this guide makes it easy.

You'll learn:

> *What each IAM policy block means.*
> *When to use it (and when not to).*
> *Real-world examples for every block.*

IAM (Identity and Access Management) is a service in AWS that helps you control access to your AWS resources. It lets you decide who can log in and what they are allowed to do. With IAM, you can make sure only the right people or services can access the resources they need—and nothing more.

IAM uses something called policies to define these permissions. Policies are written in JSON and tell AWS things like:

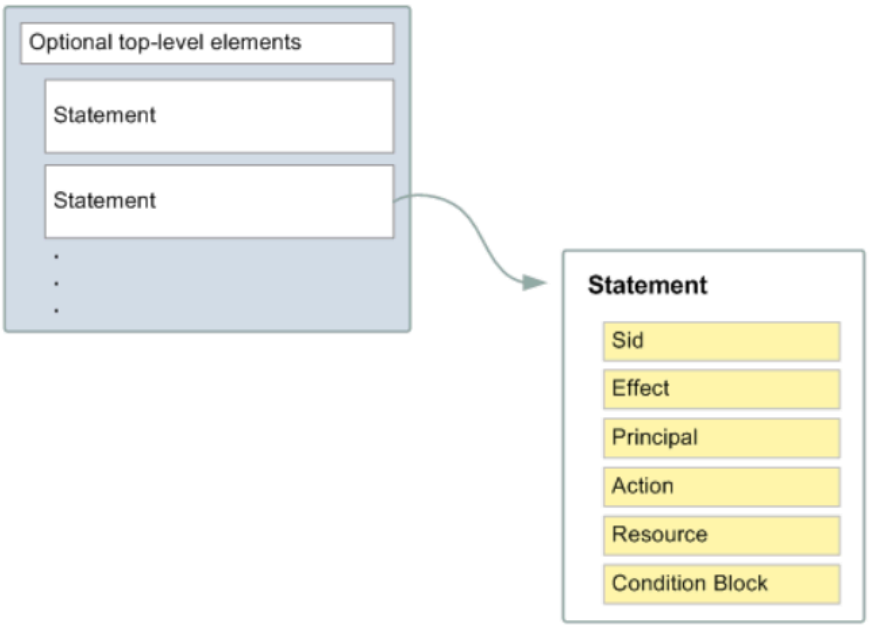
- Who can do something
- What actions they can perform
- Which resources they can access
- Under what conditions the permissions apply

Let's look at each part of an IAM policy in simple words and see how it works through examples.

1. Statement
2. Sid
3. Effect
4. Principal
5. Action

6. Resource

7. Condition



1. Statement

What It Means:

This block is the heart of your policy. It defines what permissions you're granting or denying. You can have one or more statements inside a policy.

Use It:

Always. It's required for every IAM policy.

Example:

```
"Statement": [
  {
    "Effect": "Allow",
    "Action": "ec2:DescribeInstances",
    "Resource": "*"
  }
]
```

Tip:

If you need to give different permissions to different services or actions, use multiple statement blocks.

```
"Statement": [
  {
    "Effect": "Allow",
    "Action": "s3:ListBucket",
    "Resource": "arn:aws:s3:::my-bucket"
  },
  {
    "Effect": "Deny",
    "Action": "s3:DeleteObject",
    "Resource": "arn:aws:s3:::my-bucket/*"
  }
]
```

2. `Sid` (Statement ID)

What It Means:

The `Sid` is a unique name you can give to a statement. It stands for Statement ID and helps you identify and manage specific parts of your policy.

Use It:

Helpful when policies have many statements. Useful for tracking and auditing.

Can Skip:

Yes. It's optional but recommended for clarity.

Example:

```
"Sid": "AllowListing"
```

You can use different `Sid` values for each block:

```
"Statement": [
  {
    "Sid": "ListBucketFiles",
    "Effect": "Allow",
    "Action": "s3:ListBucket",
    "Resource": "arn:aws:s3:::my-bucket"
  },
  {
    "Sid": "ReadFiles",
    "Effect": "Allow",
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::my-bucket/*"
  },
  {
    "Sid": "DenyDelete",
    "Effect": "Deny",
    "Action": "s3:DeleteObject",
    "Resource": "arn:aws:s3:::my-bucket/*"
  }
]
```

3. `Effect`

What It Means:

This defines whether the permissions are allowed or denied.

- **`"Allow"`: Grants permission**
- **`"Deny"`: Explicitly denies permission, even if other policies allow it**

Use It:

Always required. Without this, the policy will not work.

Examples:

Allow listing S3 buckets:

```
"Effect": "Allow"
```

Deny deleting objects:

"Effect": "Deny"

Tip:

Use `Deny` when you want to prevent specific actions even if a user has other permissions allowing them.

4. `Principal`

What It Means:

Defines who is allowed or denied access. This is used to specify the AWS user, role, service, or account that the policy applies to.

Only used in resource-based policies, such as those for:

> **S3 bucket policies**
> **Lambda function policies**
> **SNS topic policies**
> **IAM trust policies (used when creating roles that can be assumed)**

Use It:

Only in ***resource-based policies***.

Don't Use:

In *identity-based policies* (IAM users, roles, or groups). Identity-based policies are already **attached** to a user/role, so the "who" is already known and doesn't need a `Principal`.

Examples:

Allow a specific IAM role from another account:

```
"Principal": {
  "AWS": "arn:aws:iam::123456781234:role/DeploymentRole"
}
```

Real-world scenario: Allow a specific IAM role from another account

- You have two AWS accounts:
  Account A (where the S3 bucket lives)
  Account B (where the CI/CD server runs with the role `DeploymentRole`)

- If you want the CI/CD role in Account B to access the bucket in Account A, you must:
- **Add a resource-based policy to the S3 bucket in Account A**
  That policy will specify the role from Account B as the `Principal`.

Allow all identities (users, roles) in an AWS account:

```
"Principal": {
  "AWS": "arn:aws:iam::123456781234:root"
}
```

***Important**: This doesn't refer to the root user. It means **all identities** in that AWS account — IAM users, roles, etc.*

Allow a service (used in trust policies):

```
"Principal": {
  "Service": "ec2.amazonaws.com"
}
```

Allow a federated identity provider (e.g., Cognito):

```
"Principal": {
  "Federated": "cognito-identity.amazonaws.com"
}
```

5. Action

What It Means:

Specifies what operations are allowed or denied. These are AWS API actions like:

```
s3:GetObject
ec2:StartInstances
lambda:InvokeFunction
```

Use It:

Always. This tells AWS what exactly you're allowing/denying.

Examples:

Allow basic Lambda function use:

```
"Action": [
  "lambda:InvokeFunction"
]
```

Allow full access to EC2:

```
"Action": "ec2:*"
```

Allow read-only access to S3:

```
"Action": [
  "s3:GetObject",
  "s3:ListBucket"
]
```

6. Resource

What It Means:

Specifies the AWS resources the actions apply to. Could be a specific S3 bucket, EC2 instance, Lambda function, etc.

Use It:

Always.

Examples:

Single S3 bucket:

"Resource": "arn:aws:s3:::my-bucket"

All files in a bucket:

"Resource": "arn:aws:s3:::my-bucket/*"

All resources (use carefully):

"Resource": "*"

7. Condition

What It Means:

Adds extra filtering rules. Helps restrict access by time, IP, MFA, tags, etc.

Use It:

When you need **conditional access**.

Can Skip:

Yes. Optional but powerful.

Examples:

Allow only from specific IP:

```
"Condition": {
  "IpAddress": {
    "aws:SourceIp": "192.168.0.0/24"
  }
}
```

Require MFA:

```
"Condition": {
  "Bool": {
    "aws:MultiFactorAuthPresent": "true"
  }
}
```

Allow access only before a specific date:

```
"Condition": {
  "DateLessThan": {
    "aws:CurrentTime": "2025-12-31T23:59:59Z"
  }
}
```

complete example of an AWS IAM Role policy that includes the full structure:

Example: Give Read Access to S3 Bucket

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowReadFromIP",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::123459781234:role/MyCICDRole"
      },
      "Action": [
```

```
        "s3:GetObject"
      ],
      "Resource": "arn:aws:s3:::my-secure-bucket/*",
      "Condition": {
        "IpAddress": {
          "aws:SourceIp": "203.0.113.0/24"
        }
      }
    }
  ]
}
```

Example: Allow Lambda service to assume a role

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowLambdaAssumeRole",
      "Effect": "Allow",
      "Principal": {
        "Service": "lambda.amazonaws.com"
      },
      "Action": "sts:AssumeRole",
      "Resource": "*",
      "Condition": {
        "StringEquals": {
          "aws:SourceAccount": "123456781234"
        }
      }
    }
  ]
}
```

Example: Lambda Role with Access to S3, CloudWatch, and DynamoDB

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowS3ListAndRead",
      "Effect": "Allow",
      "Action": [
        "s3:ListBucket",
        "s3:GetObject"
      ],
      "Resource": [
        "arn:aws:s3:::my-app-config",
        "arn:aws:s3:::my-app-config/*"
      ]
    },
    {
      "Sid": "AllowSSMParameterAccess",
      "Effect": "Allow",
      "Action": [
        "ssm:GetParameter",
        "ssm:GetParameters"
      ],
      "Resource": "arn:aws:ssm:us-east-1:123456781234:parameter/app/config/*"
    }
  ]
}
```

Summary:

- `Principal`: only for resource-based policies (like S3 or trust policy)
- `Effect`, `Action`, `Resource`: always needed

- `Sid`, `Condition`: optional but helpful for clarity and security
- Use `Condition` for advanced access control (IP, time, MFA, etc.)
- Identity-based = what a user/role can do
- Resource-based = who can access the resource

==========

Identity-based vs Resource-based Policies

Identity-based: What *users/roles* can do

Resource-based: Who can access a *specific resource*

▸ Identity-based Policies

> **Attached to IAM Users, Roles, or Groups**
> **Defines what they can access**
> **Does not use Principal**

Example:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::example-bucket"
    }
  ]
}
```

▸ Resource-based Policies

Attached directly to AWS resources (e.g., S3 bucket, Lambda function)
Defines **who** (via `Principal`) can access the resource

Example:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::123456781234:role/AccessRole"
      },
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::example-bucket/*"
    }
  ]
}
```

==========